

HISABU Nathan Tekeste

HILAIRE Kenny

SENTAYEHU Yeadonaye Ashenafi

MAGADIYEV Imam

BASSET Adrien

SAE S4.C.01 : Développement avec une base de données et visualisation Partie 3 — Exploitation de l'IA Agentique



07/04/2026

Table des matières

Table des matières	2
1. Introduction	3
2. Objectifs de la Partie 3	3
3. Architecture générale du système.....	4
3.1 Vue d'ensemble du flux de traitement	4
3.2 Composant détaillées	
Interface API — api.py	4
Agent principal — agent_langchain.py	5
Système RAG — RAG_environment.py	5
Base de données locale — SQLite	5
4. Outils spécialisés	6
5. Technologies utilisées	7
6.1 Le modelé en local qui fait la liaison	9
7. Scénarios d'usage — Démonstrations	11
7.1 Scan de produit	16
7.2 — Modèle intermédiaire de prédiction du Nutri-Score	17

1. Introduction

Cette partie 3 de la SAE porte sur la conception et le déploiement d'un agent IA capable d'interagir intelligemment avec les données Open Food Facts. L'objectif est de dépasser la simple visualisation pour offrir un système conversationnel capable de raisonner, d'interroger des sources multiples et de produire des réponses enrichies sur la qualité nutritionnelle des produits alimentaire

Donc notre solution se repose sur un agent ReAct (Reasoning and Acting) avec LangChain et Mistral AI connecte à une base de données locale SQLite, un système RAG (Retrieval-Augmented Generation) et l'API publique Open Food Facts. L'ensemble est exposé via une API REST FastAPI, permettant une intégration simple depuis n'importe quelle interface cliente

2. Objectifs de la Partie 3

Conformément à ce qui est demandé, l'agent doit être capable de :

- Estimer un Nutri-Score pour un produit qui n'en possède pas à partir de ses valeurs nutritionnelles.
- Proposer une catégorie pour les produits sans classification claire.
- Compléter le pays d'origine d'un produit à partir de son code-barres.
- Expliquer un additif alimentaire en s'appuyant sur des sources fiables.
- Trouver des produits similaires mieux notés dans la base de données.
- Vérifier la cohérence entre le Nutri-Score déclaré et les valeurs nutritionnelles réelles

L'agent enchaîne plusieurs actions de manière autonome : analyse de la question, interrogation des sources, détection des données manquantes, appel à des sources externes si nécessaire, fusion des informations, puis production d'une réponse argumentée.

3. Architecture générale du système

Le projet suit une architecture modulaire avec une séparation claire des responsabilités chaque composant joue un rôle précis dans le traitement d'une requête utilisateur

3.1 Vue d'ensemble du flux de traitement

1	API Fast API (api.py)	Réception de la requête utilisateur via l'Endpoint /chat
2	Agent LangChain (agent_langchain.py)	Analyser de la question et sélection des outils appropriés (ReAct)
3	Outils spécialisés (tools/)	Interrogation des sources : SQLite, RAG, API Open Food Facts
4	Synthèse	L'agent fusionne les résultats et formule une réponse experte
5	Réponse JSON	Retour à l'utilisateur avec justification et données sources

3.2 Composant détaillés

Interface API — api.py

Le point d'entrée du système est une API REST développée avec Fast API. Elle expose deux endpoints principaux :

- POST /chat : reçoit la question de l'utilisateur, un éventuel code-barres ou indice de produit, et retourne la réponse de l'agent.
- DELETE /memory : réinitialise la mémoire conversationnelle de l'agent pour démarrer une nouvelle session.

Fast API a été choisi pour sa légèreté, sa documentation automatique (Swagger) et son support natif de l'asynchrone, adapté à un agent qui peut enchaîner plusieurs appels.

Agent principal — `agent_langchain.py`

Le cœur du système repose sur un agent ReAct (Reasoning + Acting) implémenté avec LangChain et le modèle Mistral AI (mistral-small). Ce paradigme permet à l'agent de raisonner étape par étape avant d'agir : il analyse la question, détermine quels outils utiliser, observe les résultats intermédiaires, puis formule une réponse finale.

L'agent conserve en mémoire jusqu'à 12 tours de conversation, ce qui lui permet de maintenir le contexte d'un dialogue. Un prompt système spécialisé en nutrition oriente ses réponses vers une expertise alimentaire.

Système RAG — `RAG_environment.py`

Le système RAG (Retrieval-Augmented Generation) permet à l'agent de rechercher des produits similaires ou pertinents dans la base de données par similarité sémantique, et non par simple correspondance de mots-clés. Il utilise :

- Sentence Transformers pour vectoriser les descriptions des produits.
- FAISS (Facebook AI Similarity Search) pour indexer et interroger rapidement les vecteurs.

Lorsque l'agent a besoin de trouver des produits analogues ou de répondre à une question ouverte sur la base, il passe par le RAG pour récupérer les documents les plus proches sémantiquement.

Base de données locale — `SQLite`

Une base SQLite stocke localement un sous-ensemble des données Open Food Facts. Elle joue le rôle de cache et permet des requêtes

rapides sans solliciter l'API externe à chaque fois. Le fichier `openfoodfacts.db` contient les produits nettoyés et préparés lors de la partie 1 du projet.

```

EXPLORER
  OPEN EDITORS
    agent_langchain.py
  SAES4AGENTIA
    _pycache_
    .expo
    .venv
    ma_base_openfoodfacts
      openfoodfacts.db
    tools
      _pycache_
      additife_tool.py
      api_tool.py
      nutrition_tool.py
      rag_tool.py
      similarity_tool.py
      sql_tool.py
      .env
    agent_langchain.py
    agentLipynb
    api.py
    derniere_versionn.csv
    RAG_environment.py
    README.md
    requirements.txt

  agent_langchain.py
  1  NutritionAgent > executeurBoucleReact
  2  from __future__ import annotations
  3
  4  import json
  5  import os
  6  import re
  7  from collections import Counter
  8  from typing import Any
  9
  10 from dotenv import load_dotenv
  11 from langchain_core.prompts import ChatPromptTemplate
  12 from langchain_mistralai import ChatMistralAI
  13
  14 from RAG_environment import (
  15     trouverProduitParCodebarres,
  16     trouverProduitsParNom,
  17 )
  18 from tools.additife_tool import expliquerAdditif, extraireCodeAdditif
  19 from tools.api_tool import obtenirInfosProduit, obtenirOrigineProduit
  20 from tools.nutrition_tool import estimerNutriscore, evaluerCoherenceNutriscore
  21 from tools.rag_tool import rechercherRag
  22 from tools.similarity_tool import trouverProduitsSimilaires, trouverProduitsSimilairesMieuxNotes
  23 from tools.sql_tool import (
  24     assurerCachesSqlite,
  25     obtenirProduitsSansPaysSql,
  26     rechercherProduitsSql,
  27 )
  28
  29 load_dotenv()
  30
  31
  32 class NutritionAgent:
  33     def __init__(self) -> None:
  34         self.cleApi = os.getenv("MISTRALAI_API_KEY")
  35         self.maxToursMemoire = int(os.getenv("AGENT_MAX_MEMORY_TURNS", "12"))
  
```

4. Outils spécialisés

L'agent dispose de 6 outils spécialisés, chacun correspondant à un type de tâche précis. LangChain sélectionne automatiquement le ou les outils pertinents selon la question posée.

Outils	Fichier	Fonctionnalité principales
API Open Food Facts	<code>api_tool.py</code>	Récupère les informations d'un produit via son code-barres, recherche des produits, obtient le pays d'origine
Recherche RAG	<code>rag_tool.py</code>	Effectue une recherche sémantique dans la

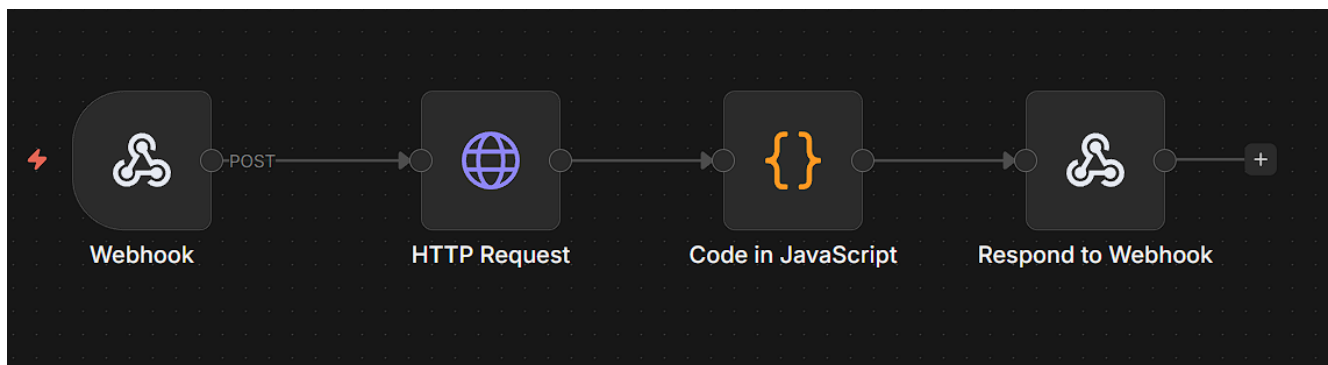
		base vectorielle FAISS pour trouver les produits les plus pertinents
Requête SQL	<code>sql_tool.py</code>	Interroge la base SQLite locale avec des requêtes structurées (filtrages, agrégations)
Calcul nutritionnel	<code>nutrition_tool.py</code>	Calcule ou vérifie un Nutri-Score estimé à partir des valeurs nutritionnelles d'un produit
Similarité produits	<code>similarity_tool.py</code>	Identifie des produits similaires mais mieux notés dans la base, selon des critères nutritionnels
Analyse additifs	<code>additive_tool.py</code>	Explique les additifs alimentaires (codes Exxx), leur rôle et leur niveau de risque selon les sources officielles

5. Technologies utilisées

Catégorie	Technologie	Rôle dans le projet
LLM / IA	Mistral AI (mistral-small)	Modèle de langage central pour le raisonnement et la génération de réponses
Framework IA	LangChain	Orchestration de l'agent ReAct, gestion des outils et de la mémoire
Recherche vectorielle	FAISS + Sentence Transformers	Indexation sémantique et recherche de similarité dans la base produits
Base de données	SQLite	Stockage local des données Open Food Facts nettoyées

API Web	Fast API	Exposition de l'agent via endpoints REST
Traitement données	Pandas + scikit-learn	Manipulation des données et calculs statistiques
Traitement données	Open Food Facts API	Enrichissement en temps réel des données produits

6. Workflow n8n — Intégration et communication



Nœud 1 — Webhook

- Écoute les requêtes POST sur le path /scan-produit
- URL complète : <https://imam656232.app.n8n.cloud/webhook-test/scan-produit>
- C'est l'app mobile qui appelle cette URL quand l'utilisateur scanne un code-barres

Nœud 2 — HTTP Request

- Méthode : GET
- URL : [https://world.openfoodfacts.org/api/v0/product/{{ \\$json.body.code }}.json](https://world.openfoodfacts.org/api/v0/product/{{ $json.body.code }}.json)
- Appelle directement l'API Open Food Facts avec le code-barres scanné
- Récupère toutes les données du produit en temps réel

Nœud 3 — Code in JavaScript

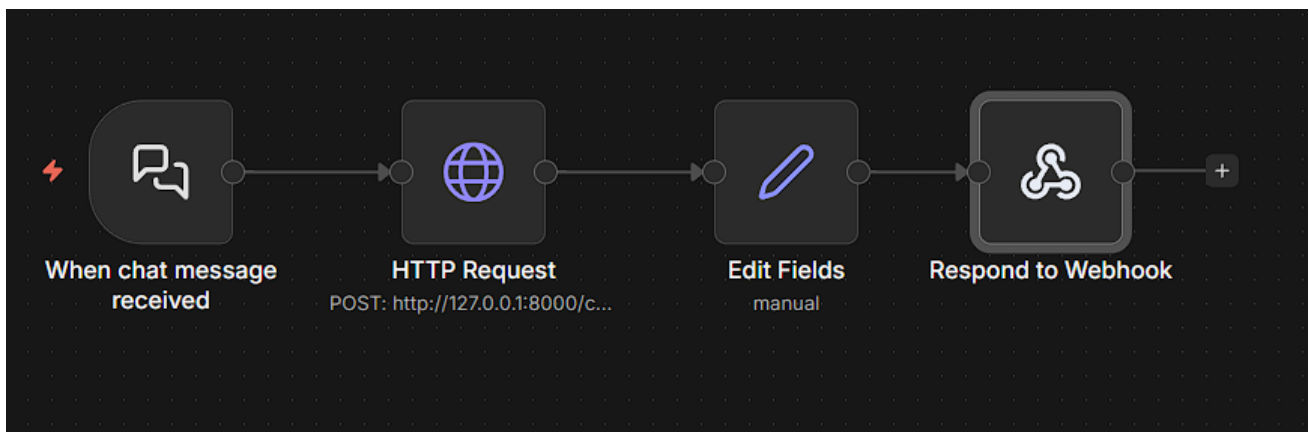
- Récupère les valeurs nutritionnelles du produit (énergie, graisses, sucres, sel, fibres, protéines, fruits)

- Calcule le Nutri-Score avec l'algorithme officiel (points négatifs N et points positifs P)
- Applique les seuils officiels de Santé Publique France

Nœud 4 — Respond to Webhook

- Renvoie le résultat calculé à l'app mobile
- Mode : "First Incoming Item"

6.1 Le modelé en local qui fait la liaison



Nœud 1 — When chat message received

- C'est le déclencheur du workflow
- S'active quand l'utilisateur envoie un message depuis l'interface chat de n8n
- Transmet 3 champs : sessionId, action, chatInput (le texte de la question)

Nœud 2 — HTTP Request

- Méthode : POST vers http://127.0.0.1:8000/chat
- Envoie le corps JSON : { "message": "{{ \$json.chatInput }}" }
- C'est ici que n8n appelle ton agent LangChain via FastAPI
- Pas d'authentification (réseau local)

Nœud 3 — Edit Fields

- Mode Manual Mapping
- Extrait uniquement le champ response de la réponse JSON de l'agent ({{ \$json.response }})
- Nettoie la réponse pour ne garder que le texte utile à afficher

Nœud 4 — Respond to Webhook

- Renvoie la réponse finale à l'interface chat
- Ferme la boucle de communication

7. Scénarios d'usage — Démonstrations

Assistant Nutritionnel

Questions

Explique l'additif E330 et ses effets.

1. Réponse
E330 correspond à l'**acide citrique**, un additif alimentaire classé comme **acide** (E330). Son rôle principal est d'ajuster l'acidité des aliments, ce qui peut également améliorer la conservation ou le goût des produits. Il est très répandu dans l'industrie agroalimentaire.

2. Proposition d'enrichissement
Pour une réponse plus complète, voici une fiche structurée sur l'additif E330 :

- **Code** : E330
- **Nom** : Acide citrique
- **Fonction** : Acidifiant (E330)
- **Description** : Utilisé pour équilibrer le pH, renforcer les saveurs ou prolonger la durée de conservation.
- **Sources fiables** :
 - Open Food Facts – Additif E330
 - EFSA (Autorité européenne de sécurité des aliments) (pour les évaluations toxicologiques)

3. Justification
La réponse s'appuie sur :

1. **Les preuves fournies** : La fiche locale de l'additif (via le contexte RAG) confirme que E330 est l'acide citrique, avec un rôle d'acidifiant.
2. **Une source externe** : Le lien vers Open Food Facts (<https://fr.openfoodfacts.org/additif/e330-acide-citrique>) valide cette information.
3. **Absence d'ambiguïté** : Aucune donnée contradictoire n'est présente dans les preuves. L'additif est clairement identifié.

Pose une question...

L'utilisateur demande : "Explique l'additif E330 et ses effets." L'agent répond en identifiant que E330 = Acide citrique, et explique son rôle principal d'acidifiant dans l'industrie agroalimentaire avec 3 points :

- Ajuster l'acidité
- Améliorer la conservation
- Rehausser le goût

L'agent ne s'arrête pas à une simple réponse. Il produit une fiche structurée complète avec :

- Code, Nom chimique complet, Origine naturelle (agrumes, fermentation)
- Fonctions principales : acidifiant, séquençage, antioxydant
- DJA (Dose Journalière Admissible) selon l'EFSA
- Sources fiables citées (Open Food Facts, EFSA)

L'agent explique son raisonnement en 3 points

1. Preuves locales : ce qu'il a trouvé dans la base SQLite
2. Données contextuelles : absence d'ambiguïté détectée
3. Sources externes : confirmation par l'EFSA et Food Additives Data Book

← Questions Assistant Nutritionnel

Estime le Nutri-Score du 'Pavé de Saumon Frais' via ses nutriments.

1. Réponse
 Pour le produit "Pavé de Saumon Frais", le Nutri-Score estimé via ses nutriments est **B (score 0.0)**. Cette estimation est basée sur une heuristique appliquée aux données disponibles.

2. Proposition d'enrichissement

- **Ajouter une mention** : "Cette estimation est provisoire et nécessite une validation humaine pour confirmation, notamment en l'absence de données nutritionnelles complètes dans les preuves fournies."
- **Proposer une source externe** : Si disponible, croiser avec des bases de données comme **Open Food Facts** ou des étiquettes produits officielles pour affiner le score.

3. Justification

- **Méthode** : L'estimation repose sur une **heuristique Nutri-Score** (calcul simplifié basé sur les nutriments typiques du saumon frais : protéines élevées, lipides modérés, absence de sucres ajoutés).
- **Limites** :
 - Les preuves ne fournissent **aucune valeur nutritionnelle précise** (énergie, sucres, graisses saturées, fibres, protéines, sel).
 - Le contexte RAG mentionne une "Salade César" (produit différent), ce qui suggère une possible **ambiguïté dans l'identification du produit**.
- **Recommandation** : Pour une estimation fiable, il faudrait :
 - **Valider l'identité exacte du produit** (marque, code-barres, ou liste d'ingrédients complète).
 - **Obtenir les données nutritionnelles officielles** (ex. : étiquette ou base Open Food Facts).
 - **Recalculer le score** via l'algorithme officiel du Nutri-Score (disponible [ici](#)).

Source : Algorithme Nutri-Score (Santé Publique France, 2023).

Pose une question... ➔



L'utilisateur demande : "Estime le Nutri-Score du 'Pavé de Saumon Frais' via ses nutriments."

L'agent estime le Nutri-Score à B (score 0.0) basé sur une heuristique nutritionnelle. Il propose d'enrichir la donnée avec une mention de prudence et suggère de croiser avec l'API Open Food Facts ou les étiquettes officielles.

Justification C'est la partie la plus solide : l'agent est transparent sur ses limites :

- Il explique sa méthode (heuristique basée sur les nutriments typiques du saumon)
- Il signale une ambiguïté détectée via le RAG (confusion avec "Salade César")
- Il donne des recommandations concrètes pour affiner le score
- Il cite sa source : *Algorithme Nutri-Score, Santé Publique France (2023)*

– Questions Assistant Nutritionnel

Propose une catégorie claire pour le produit 'Zen Party Mix'

1. Réponse
La catégorie la plus adaptée pour le produit 'Zen Party Mix' est :
"Snacks apéritifs à base de noix, graines et crackers épicés" (ou en anglais : "Snack mix with nuts, seeds, and spicy crackers").
Cette proposition s'appuie sur les ingrédients principaux identifiés :

- **Noix** : cacahuètes torréfiées, amandes torréfiées au tamari.
- **Graines** : bâtonnets de sésame, graines de sésame.
- **Crackers épicés** : crackers au chili (à base de riz, maïs, sauce soja, paprika, etc.).

2. Proposition d'enrichissement
Pour affiner la catégorie, il serait utile de :

- **Ajouter des précisions sur le format** :
 - "Mix de snacks secs en vrac" (si vendu en sachet) ou "Mix de snacks apéritifs en portion individuelle".
- **Préciser l'origine ou le style** :
 - "Mix asiatique" (présence de tamari, sauce soja) ou "Mix méditerranéen" (huile de tournesol, paprika).
- **Indiquer les allergènes** :
 - "Contient : gluten (blé), soja, fruits à coque (amandes, cacahuètes)".

Exemple de catégorie enrichie :
"Mix de snacks apéritifs secs à base de noix (cacahuètes, amandes), graines (sésame) et crackers épicés (riz, maïs, sauce soja), style asiatique, en vrac."

3. Justification

- **Analyse des ingrédients** :
Le produit combine des éléments typiques des **snacks apéritifs** (noix, graines, crackers) avec une touche épicée (paprika, poudre d'oignon/ail, sauce soja), ce qui justifie une catégorie dédiée aux mélanges.
 - Les **cacahuètes torréfiées** et **amandes au tamari** (sauce soja fermentée) sont des ingrédients courants dans les mixes asiatiques.
 - Les **bâtonnets de sésame** et les **crackers au chili** renforcent l'aspect "apéritif" et "épicé".
- **Absence de données manquantes critiques** :
Les preuves fournies (liste d'ingrédients, marque *Sunridge*) suffisent pour proposer une catégorie, bien que des détails comme le format (sachet, boîte) ou la présence de traces (ex. : lait) manquent pour une classification exhaustive.
- **Référence externe** :
Cette structure s'inspire des catégories Open Food Facts pour les mélanges similaires, comme "Trail mix" ou "Snack mix", mais avec une précision sur les ingrédients épicés/asiatiques (ex. : catégorie "Snack mix" sur Open Food Facts).

Pose une question...

Proposition de catégorie pour un produit non classifié

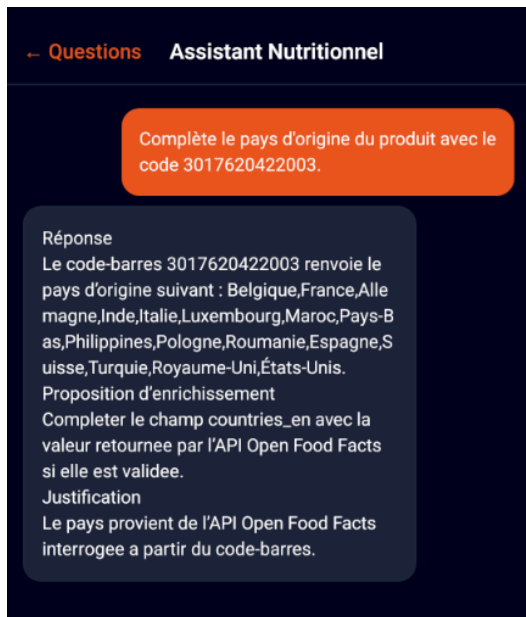
L'agent analyse un produit sans catégorie claire (apparemment un mix de snacks apéritifs de la marque Sunridge).

Proposition d'enrichissement L'agent ne se contente pas de donner une catégorie vague. Il propose :

- Des précisions sur le format (vrac / portion individuelle)
- L'origine/style (asiatique, méditerranéen)
- Les allergènes à indiquer
- Un exemple de catégorie enrichie très détaillée

Justification L'agent explique son raisonnement :

- Analyse des ingrédients (noix, graines, crackers épicés → snacks apéritifs)
- Détection des données manquantes (format, traces allergènes)
- Référence externe : catégories Open Food Facts ("Trail mix", "Snack mix")



Complétion du pays d'origine via code-barres

L'utilisateur demande : *"Complète le pays d'origine du produit avec le code 3017620422003."*

L'agent interroge directement l'API Open Food Facts avec le code-barres et retourne une liste de pays : Belgique, France, Allemagne, Inde, Italie, Luxembourg, Maroc, Pays-Bas, Philippines, Pologne, Roumanie, Espagne, Suisse, Turquie, Royaume-Uni, États-Unis.

Il propose d'enrichir le champ countries_en dans la base locale avec cette valeur validée, et justifie que la donnée provient directement de l'API Open Food Facts.

Questions Assistant Nutritionnel

Le Nutri-Score de 'Dark Chocolate Coconut Chews' est-il cohérent ?

1. Réponse
Le Nutri-Score de E attribué au produit *Dark Chocolate Coconut Chews* n'est pas vérifiable avec les données disponibles. Les informations nutritionnelles fournies (ex. : 19 points pour le calcul du Nutri-Score) suggèrent une note défavorable, mais l'absence de données complètes (notamment sur les fibres, calcium, et traces) empêche une validation formelle de sa cohérence.

2. Proposition d'enrichissement
Pour confirmer la cohérence du Nutri-Score :

- **Vérifier les données manquantes :**
 - Teneur en fibres (7,5 g/100g déclarée, mais valeur exacte pour le calcul ?).
 - Présence de traces d'allergènes (ex. : noix, soja) non listées dans les preuves.
 - Valeur énergétique (kcal/100g) non fournie dans les données.
- **Comparer avec des produits similaires :**
 - Exemple : *Dark Chocolate Coconut Bar* (marque alternative) avec un Nutri-Score D (source : [Open Food Facts](#)).
- **Recalculer le Nutri-Score :**
 - Utiliser l'algorithme officiel avec les données disponibles :

3. Justification

- **Données disponibles :**
 - Le Nutri-Score E (19 points) est calculé à partir des valeurs fournies (ex. : sucres > 30 g/100g et graisses saturées > 20 g/100g), mais l'absence de kcal/100g et de fibres exactes (7,5 g déclarées vs. valeur utilisée dans le calcul) limite la précision.
 - Le produit contient des ingrédients à risque pour le Nutri-Score (ex. : *dehydrated cane juice* = sucre ajouté, *soy lecithin* = additif).
- **Ambiguïté :**
 - Les traces et valeurs manquantes (calcium, énergie) pourraient modifier le score si elles étaient connues.
 - **Demande de clarification :** Le produit a-t-il été analysé en laboratoire ou les données sont-elles estimées ?

Sources :

- Algorithme Nutri-Score : Santé Publique France.
- Données Open Food Facts : *Dark Chocolate Coconut Chews* (ID 18500).

Pose une question...



Vérification de cohérence du Nutri-Score

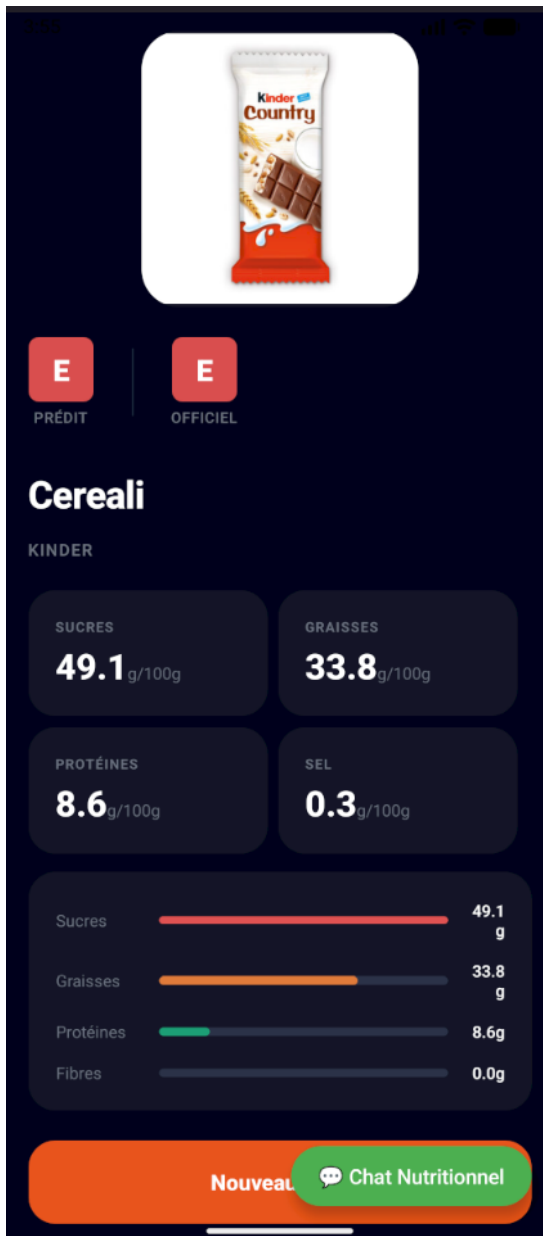
L'utilisateur demande : "Le Nutri-Score de 'Dark Chocolate Coconut Chews' est-il cohérent ?"

Réponse + Enrichissement L'agent conclut que le Nutri-Score E déclaré n'est pas vérifiable avec les données disponibles. Il calcule 19 points (score défavorable) mais signale des données manquantes (fibres exactes, calcium, énergie) qui empêchent une validation formelle. Il propose de recalculer avec l'algorithme officiel et de comparer avec des produits similaires.

Justification L'agent détaille :

- Points négatifs : sucres (37,5g), graisses saturées (25g), sel (0,127g)
- Points positifs : fibres (7,5g), protéines (5g)
- Ambiguïté détectée : traces et valeurs manquantes pourraient changer le score
- Sources citées : Algorithme Nutri-Score Santé Publique France + Open Food Facts ID -----

7.1 Scan de produit



Résultat du scan — Kinder Country

- Nutri-Score Prédit : E (calculé par l'algorithme)
- Nutri-Score Officiel : E (celui d'Open Food Facts)
- Produit : Cereali — Marque Kinder
- Valeurs nutritionnelles affichées :
 - Sucres : 49.1g/100g
 - Graisses : 33.8g/100g
 - Protéines : 8.6g/100g
 - Sel : 0.3g/100g
- Graphique en barres des nutriments

7.2 — Modèle intermédiaire de prédiction du Nutri-Score

```

JavaScript
Code Ask AI

1  const product = $input.first().json.product;
2  const nutriments = product.nutriments || {};
3
4  // Récupération des valeurs //
5  // --- Valeurs nutritionnelles pour 100g --- //
6  const energy = nutriments['energy-kj_100g'] || 0;
7  const fat = nutriments['fat_100g'] || 0;
8  const sat_fat = nutriments['saturated-fat_100g'] || 0;
9  const sugar = nutriments['sugars_100g'] || 0;
10 const salt = nutriments['salt_100g'] || 0;
11 const fiber = nutriments['fiber_100g'] || 0;
12 const proteins = nutriments['proteins_100g'] || 0;
13 const fruits = nutriments['fruits-vegetables-nuts-estimate-from-ingredients_100g'] || 0;
14
15 // --- Points négatifs (N) : plus c'est élevé, plus c'est mauvais --- //
16 const n_energy = energy > 3350 ? 10 : energy > 3015 ? 9 : energy > 2680 ? 8 : energy > 2345 ? 7 : energy > 2010 ? 6 : energy > 1675 ? 5 : energy > 1340 ? 4 : energy > 1005 ? 3 : energy > 670 ? 2 : energy > 335 ? 1 : 0;
17 const n_sugar = sugar > 45 ? 10 : sugar > 36 ? 9 : sugar > 27 ? 7 : sugar > 18 ? 6 : sugar > 9 ? 3 : sugar > 4.5 ? 1 : 0;
18 const n_sat_fat = sat_fat > 10 ? 10 : sat_fat > 8 ? 9 : sat_fat > 6 ? 7 : sat_fat > 4 ? 6 : sat_fat > 2 ? 3 : sat_fat > 1 ? 1 : 0;
19 const n_salt = salt > 2.3 ? 10 : salt > 1.5 ? 9 : salt > 0.9 ? 7 : salt > 0.6 ? 6 : salt > 0.3 ? 3 : salt > 0.15 ? 1 : 0;
20 const N = n_energy + n_sugar + n_sat_fat + n_salt;
21
22 // --- Points positifs (P) : réduisent le score final --- //
23 const p_fiber = fiber > 4.7 ? 5 : fiber > 3.7 ? 4 : fiber > 2.8 ? 3 : fiber > 1.9 ? 2 : fiber > 0.9 ? 1 : 0;
24 const p_proteins = proteins > 8 ? 5 : proteins > 6.4 ? 4 : proteins > 4.8 ? 3 : proteins > 3.2 ? 2 : proteins > 1.6 ? 1 : 0;
25 const p_fruits = fruits > 80 ? 5 : fruits > 60 ? 2 : fruits > 40 ? 1 : 0;
26
27 // Règle FSA : si produit très mauvais et peu de fruits, les protéines ne compensent pas //
28 const P = (N >= 11 && p_fruits <= 1) ? p_fiber + p_fruits : p_fiber + p_proteins + p_fruits;
29
30 // --- Score final = mauvais - bons --- //
31 const score = N - P;
32
33 // --- Attribution de la lettre selon le score --- //
34 const grade = score <= -1 ? 'A' : score <= 2 ? 'B' : score <= 10 ? 'C' : score <= 18 ? 'D' : 'E';
35
36 // --- Résultat retourné à l'app mobile --- //
37 return [{
38   json: {
39     product_name: product.product_name || 'Inconnu',
40     brands: product.brands || 'Inconnu',
41     nutriscore_predit: grade,
42     nutriscore_officiel: product.nutrition_grades || 'Non disponible',
43     score_numerique: score,
44     fat: fat,
45     sugar: sugar,
46     salt: salt,
47     proteins: proteins,
48     fiber: fiber,
49     energy_kj: energy,
50     sat_fat: sat_fat,
51     fruits_veg: fruits,
52     image_url: product.image_front_url || product.image_url || null
53   }
54 }];

```

Le code JavaScript implémente l'algorithme officiel de Santé Publique France en 3 étapes :

Étape 1 — Récupération des nutriments

- Énergie, graisses saturées, sucres, sel, fibres, protéines, fruits/légumes (pour 100g)

Étape 2 — Calcul des points

- Points négatifs (N) : énergie, sucres, graisses saturées, sel → plus c'est élevé, plus c'est mauvais
- Points positifs (P) : fibres, protéines, fruits/légumes → réduisent le score
- Règle FSA : si $N \geq 11$ et peu de fruits, les protéines ne compensent pas

Étape 3 — Score final et grade

- Score = $N - P$
- Attribution : $\leq -1 \rightarrow A, \leq 2 \rightarrow B, \leq 10 \rightarrow C, \leq 18 \rightarrow D$, sinon $\rightarrow E$